Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int countzeroes(int a[], int low, int high){
    if(low==high){
        if(a[low]==0)
        return 1;
        else
        return 0;
    }
    int mid=(low+high)/2;
    return countzeroes(a,low,mid)+
    countzeroes(a,mid+1,high);
}
int main(){
    int m;
    scanf("%d",&m);
    int a[100];
    for(int i=0;i<m;i++){
        scanf("%d",&a[i]);
    }
        printf("%d",countzeroes(a,0,m-1));
        return 0;

}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔

	Input	Expected	Got	
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
int majority(int a[],int left,int right){
    if(left==right)
    return a[left];
    int mid=(left+right)/2;
    int leftmajority=majority(a,left,mid);
    int rightmajority=majority(a,mid+1,right);
    if(leftmajority==rightmajority)
    return leftmajority;
    int leftcount=0;
    int rightcount=0;
    for(int i=left;i<=right;i++)
    {
        if(a[i]==leftmajority)
        leftcount++;
        if(a[i]==rightmajority)
        rightcount++;
    }
    if(leftcount > rightcount)
    return leftmajority;
    else
    return rightmajority;
}
int main(){
    int n;
    int a[50000];
```

```
27      scanf("%d",&n);
28      for(int i=0;i<n;i++){
29          scanf("%d",&a[i]);
30      }
31      printf("%d",majority(a,0,n-1));
32      return 0;
33  }
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int findfloor(int a[],int low,int high,int x){
    if(low>high)
    return -1;
    int mid=(low+high)/2;
    if(a[mid]==x)
    return a[mid];
    if(a[mid]<x){
        int result=findfloor(a,mid+1,high,x);
        if(result==-1)
        return a[mid];
        else
        return result;
    }
    return findfloor(a,low,mid-1,x);
}
int main(){
    int n,x;
    int a[100];
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&a[i]);
    }
    scanf("%d",&x);
    printf("%d",findfloor(a,0,n-1,x));
    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔

	Input	Expected	Got	
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int main(){
    int n,x;
    int a[100];
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        scanf("%d",&a[i]);
    }
    scanf("%d",&x);
    for(int i=0;i<n;i++){
        for(int j=i+1;j<n;j++){
            if(a[i]+a[j]==x){
                printf("%d\n",a[i]);
                printf("%d",a[j]);
                return 0;
            }
        }
    }
    printf("No");
    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>
void quicksort(int a[],int low,int high){
    if(low < high){
        int pivot=a[high];
        int i=low-1;
        for(int j=low;j<high;j++){
            if(a[j] < pivot){
                i++;
                int temp=a[i];
                a[i]=a[j];
                a[j]=temp;
            }
        }
        int temp=a[i+1];
        a[i+1]=a[high];
        a[high]=temp;
        int p=i+1;
        quicksort(a,low,p-1);
        quicksort(a,p+1,high);
    }
}
int main(){
    int n;
    scanf("%d",&n);
    int a[n];
    for(int i=0;i<n;i++){
        scanf("%d",&a[i]);
    }
    quicksort(a,0,n-1);
    for(int i=0;i<n;i++){
        printf("%d ",a[i]);
    }
    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔

	Input	Expected	Got	
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course